A Multi-Agent System for Emergency Department Management

Matteo Ferrari 5174687
5174687@studenti.unige.it

Contents

1 Details of the Proposal **2**
1.1 Full Specification of the Proposal . . . 2
1.2 The Difficulty of The Proposal . . . 2

2 Domain Model (Ontology) **3**
2.1 Simulation Time Scale . . . 3
2.2 Severity Levels . . . 3
2.3 Triage Colors . . . 3
2.4 PatientQueueEntry . . . 3
2.5 BedInfo . . . 4

3 Agents and Artifacts **4**
3.1 AutomaticSpawnAgent . . . 4
3.2 RegistrationAgent . . . 4
3.3 TriageAgent . . . 4
3.4 QueueManagerArtifact . . . 5
3.5 BedManagerArtifact . . . 5
3.6 DoctorAgent . . . 5
3.7 NurseAgent . . . 6
3.8 AmbulanceAgent . . . 6
3.9 PatientAgent . . . 6

4 System Architecture and Workflow **7**
4.1 Patient Intake and Triage . . . 7
4.2 Medical Examination and Treatment . . . 8
4.3 Monitoring and Re-evaluation Loop . . . 8

5 User Interface **8**
5.1 Configuration Interface . . . 8
5.2 Main Simulation Dashboard . . . 9

6 Code Specification and Implementation **13**
6.1 Development Stack and Libraries . . . 13
6.2 Concurrency and Synchronization . . . 13
6.3 Project Structure and Packages . . . 13
6.4 Metrics and Repository . . . 14
6.5 Use of Artificial Intelligence and LLMs . . . 14

7 Future Improvements **14**

1 Details of the Proposal

The project consists of the design and implementation of a Multi-Agent System (MAS) for the management and simulation of a hospital Emergency Department. The goal is to model the clinical workflow and the interactions between autonomous agents representing the main actors and resources of the department.

1.1 Full Specification of the Proposal

The following section presents the original description submitted in the initial proposal form. This served as the conceptual foundation for the project, though the system was significantly expanded and refactored during the actual development phase to better reflect real-world dynamics.

The project consists of the design and implementation of a Multi-Agent System for the management and simulation of an Emergency Department. The goal is to model the primary actors involved in an Emergency Department as autonomous agents and enable them to interact with one another.
The project is inspired by the paper "An Agent-Based Simulation Tool for Scheduling Emergency Department Physicians" [4].
The system will be composed of the following agents:

- 1 Triage Agent
- 1 Queue Manager Agent
- 1 Bed Manager Agent
- n Bed Agents
- n Doctor Agents
- 1 Ambulance Agent
- n Patient Agents

During the implementation, several architectural improvements and functional expansions were introduced. The main differences compared to the initial plan are:

- **Transition to Artifacts:** The **Queue Manager** and **Bed Manager** are not implemented as agents but as **Artifacts**. Consequently, the individual **Bed Agents** have been integrated into a centralized dictionary within the **Bed Manager Artifact**.
- **Registration Agent:** I introduced a dedicated agent responsible for the initial intake and registration of walk-in patients.
- **Enhanced Ambulance Fleet:** The system supports multiple **Ambulance Agents**.
- **Automatic Spawn Agent:** To ensure a dynamic simulation, I added an agent responsible for generating new patients at randomized intervals.
- **Nurse Agents:** A configurable number of **Nurse Agents** was implemented to manage periodic patient re-evaluation and bed monitoring.

1.2 The Difficulty of The Proposal

The project is a creative project, and is classified as **hard** because it requires the design and implementation of a complex Multi-Agent System, with many different types of agents and many interactions between them.

2 Domain Model (Ontology)

This section describes the domain model of the system, including the main concepts and entities involved in Emergency Department (ED) management.

2.1 Simulation Time Scale

To balance the need for a dynamic real-time visualization with the realistic duration of medical procedures, the simulation adopts a scaled time model. Specifically, the time ratio is set such that **1 second** of simulation time corresponds to **2 minutes** of real-world Emergency Department activity. This scaling ensures that the treatment cycles and patient flows are observable within a reasonable testing timeframe while maintaining proportional clinical durations.

2.2 Severity Levels

Severity levels are used to classify patients according to the urgency of their conditions. Three levels are defined:

- **Low**
- **Medium**
- **High**

Each severity level is assigned to a patient either randomly at spawn time by the **AutomaticSpawnAgent** (see Section 3.1) or manually by the user through the corresponding GUI control.

2.3 Triage Colors

Triage colors are used to classify patients based on clinical severity and to determine queue priority. Five colors are considered, based on the triage system currently used in Italy:

- **White** (Priority 0)
- **Green** (Priority 1)
- **Blue** (Priority 2)
- **Orange** (Priority 3)
- **Red** (Priority 4)

Each color is mapped to a numerical value to facilitate sorting within the **QueueManagerArtifact** (see Section 3.4).

2.4 PatientQueueEntry

The `PatientQueueEntry` is a data structure representing an entry in the priority queue. It contains the following fields:

- `patientId`: the unique identifier of the patient.
- `entryColor`: the initial triage color assigned to the patient. It may differ from the current color and is primarily used for calculating discharge time statistics.
- `arrivalTime`: the timestamp of the patient's arrival in the ED.

2.5 BedInfo

The `BedInfo` structure stores information regarding a specific bed. It contains:

- `bedId`: the unique identifier of the bed.
- `patientId`: the ID of the patient currently occupying the bed (null if available).
- `color`: the current triage color of the occupant.
- `admissionTime`: the time at which the patient was admitted to the bed.
- `queueEntry`: a reference to the `PatientQueueEntry` of the occupant.

Note: Although `queueEntry` contains redundant information (such as the `patientId`), it was maintained to simplify the update of statistics upon discharge without requiring a full refactor of the existing logic.

3 Agents and Artifacts

This section provides a detailed description of the **Agents** and **Artifacts** that compose the system, outlining their specific behaviors and roles. For a high-level overview of how these components interact throughout a patient's journey, please refer to the system architecture and workflow diagram in Section 4.

3.1 AutomaticSpawnAgent

The **AutomaticSpawnAgent** is responsible for the continuous generation of new patients in the Emergency Department at random intervals, governed by a configurable maximum time between spawns. When an instance of **PatientAgent** (Section 3.9) is created, it is assigned a random severity level. Depending on this level and the real-time availability of an **AmbulanceAgent** (Section 3.8), the patient will either arrive as a walk-in case or be transported by ambulance. This initial branching significantly impacts the patient's subsequent flow and interaction within the system.

3.2 RegistrationAgent

The **RegistrationAgent** is responsible for registering patients in the Emergency Department. This agent interacts with incoming walk-in patients by receiving an `ACL Message` from the **PatientAgent** with a registration request. The **RegistrationAgent** replies with another `ACL Message` confirming the registration or with a "*Wait*" message if the registration desk is currently busy with another patient.
While the registration desk is busy, the **PatientAgent** waits in a registration queue.
After the registration is completed, the **RegistrationAgent** forwards the patient's data to the **TriageAgent** (Section 3.3) via an `ACL Message` containing the ID and the severity level of the patient.

3.3 TriageAgent

The **TriageAgent** is responsible for assigning a triage color to each patient based on their initial severity level.
The color is assigned according to the following mapping:

- **Low severity:** White, Green, or Blue.
- **Medium severity:** Blue or Orange.
- **High severity:** Orange or Red.

Each color is assigned based on a specific probability distribution, defined during the design phase to reflect typical clinical scenarios.
After assigning the triage color, the **TriageAgent** sends an `ACL Message` to the patient containing the assigned color and then forwards the patient's data to the **QueueManagerArtifact**.
Similarly to the **RegistrationAgent**, the **TriageAgent** can only process one patient at a time. Therefore, incoming patients must wait in a triage-specific queue until they receive a confirmation from the agent.

3.4 QueueManagerArtifact

The **QueueManagerArtifact** is responsible for managing the priority queue of patients waiting to be examined by a doctor. The queue order is strictly determined by the triage color assigned to each patient. The artifact stores both the `PatientQueueEntry` and the current triage color for each patient.
Originally, the Queue Manager was implemented as an agent. In that version, it managed the queue through asynchronous messages between the **TriageAgent**, the **RegistrationAgent**, and the **DoctorAgent** instances (Section 3.6). The **RegistrationAgent** would add a patient to the queue without a color, and the **TriageAgent** would later send a message to update the color and the patient's priority.
While functional, this agent-based implementation proved overly complex. Specifically, when multiple **DoctorAgent** instances were introduced, a *race condition* occurred: if two or more doctors requested a patient simultaneously, they could occasionally be assigned the same one. Attempting to have the Queue Manager "push" patients to available doctors also failed, as it led to inefficient workload distribution, where some doctors remained idle while others were overloaded.
To resolve these issues, the **QueueManager** was refactored from an **Agent** into an **Artifact**. This allows doctors to access the queue directly through operations protected by a **lock mechanism**, effectively eliminating race conditions. Additionally, the system flow was simplified by removing the connection between the **RegistrationAgent** and this **Artifact**, leaving the **TriageAgent** as the sole entity responsible for adding patients to the queue once triage is complete.

3.5 BedManagerArtifact

The **BedManagerArtifact** is responsible for managing the beds within the Emergency Department, maintaining a real-time registry of occupied and available slots. It utilizes a `HashMap` that maps each `bedId` to a `BedInfo` data structure, which stores all relevant details about the bed and its current occupant.
The artifact provides methods to check bed availability, assign a bed to a patient, and release a bed upon discharge. When a patient requires hospitalization, the **DoctorAgent** queries the **BedManagerArtifact**. If a bed is available, the patient is admitted immediately. However, if the department is at full capacity, the artifact performs a priority-based replacement: it identifies the patient with the lowest triage priority; if multiple patients share the same lowest priority, the one with the longest stay is discharged to accommodate the new patient.
Following the successful refactoring of the **QueueManagerArtifact**, the Bed Manager was also implemented as an **Artifact** rather than an agent. As a shared resource accessed simultaneously by multiple **DoctorAgent** and **NurseAgent** (Section 3.7) instances, managing access via a lock mechanism proved more efficient than an asynchronous messaging system, which would have significantly increased system overhead and architectural complexity.

3.6 DoctorAgent

The **DoctorAgent** is responsible for treating patients within the Emergency Department, representing an abstract model of medical examination and treatment. In this simulation, all

doctor instances share the same behavior without specialization. Their operational logic is as follows:

- **Patient Retrieval:** When available, a doctor attempts to retrieve the next patient from the **QueueManagerArtifact**. If the queue is empty, the doctor remains in an idle state until a new patient is added.
- **Treatment Simulation:** Upon retrieving a patient, the doctor performs a treatment that lasts for a random duration within a range (minimum to maximum) defined for each triage level. These time ranges were calibrated during development to simulate realistic hospital dynamics.
- **Condition Improvement:** Following treatment, the patient's triage color may decrease by one or two levels (representing an improvement in their condition) or remain unchanged. This transition is governed by a probability distribution defined to reflect successful medical intervention.
- **Discharge or Hospitalization:** If the post-treatment color is **White**, the patient is immediately discharged. For other colors (e.g., **Green** or **Blue**), the doctor evaluates the need for hospitalization based on a color-specific probability. If hospitalization is required, the doctor attempts to assign a bed through the **BedManagerArtifact**.

3.7 NurseAgent

The **NurseAgent** is responsible for monitoring the conditions of hospitalized patients and managing their periodic re-evaluation.
The number of **NurseAgent** instances is configurable by the user at the start of the simulation, with the constraint that it must be less than or equal to the total number of beds. To optimize efficiency, the available beds are distributed among the nurses; each agent is responsible for a specific subset of beds, ensuring an equal distribution of the workload.
The core behavior of the **NurseAgent** consists of a periodic monitoring loop. For each assigned bed, the nurse monitors the occupant's status and, after a predefined interval, autonomously re-inserts the patient into the **QueueManagerArtifact**. This mechanism ensures that hospitalized patients are consistently re-queued for medical follow-up by a **DoctorAgent**, allowing for potential triage color updates or discharge.

3.8 AmbulanceAgent

The **AmbulanceAgent** is responsible for transporting patients with **Medium** or **High** severity levels to the Emergency Department.
In addition to transportation, this agent simulates the pre-hospital registration process.
Unlike walk-in patients, those arriving via ambulance bypass the **RegistrationAgent** and are forwarded directly to the **TriageAgent**.

3.9 PatientAgent

The **PatientAgent** represents an individual patient within the Emergency Department. It acts as a reactive entity, responsible for interacting with all other agents and artifacts throughout its journey from arrival to final discharge.
This agent maintains its own internal state, including its assigned severity level and current triage color. It manages communication by sending registration requests to the **RegistrationAgent** and responding to instructions from the **TriageAgent** and **DoctorAgent**. The **PatientAgent** remains active in the system until the treatment process is complete or it is discharged from a bed by the **BedManagerArtifact**.

4 System Architecture and Workflow

The overall architecture of the system and the interactions among agents and artifacts are illustrated in Figure 1. The workflow represents a complete lifecycle of a patient within the Emergency Department, from initial arrival to final discharge.

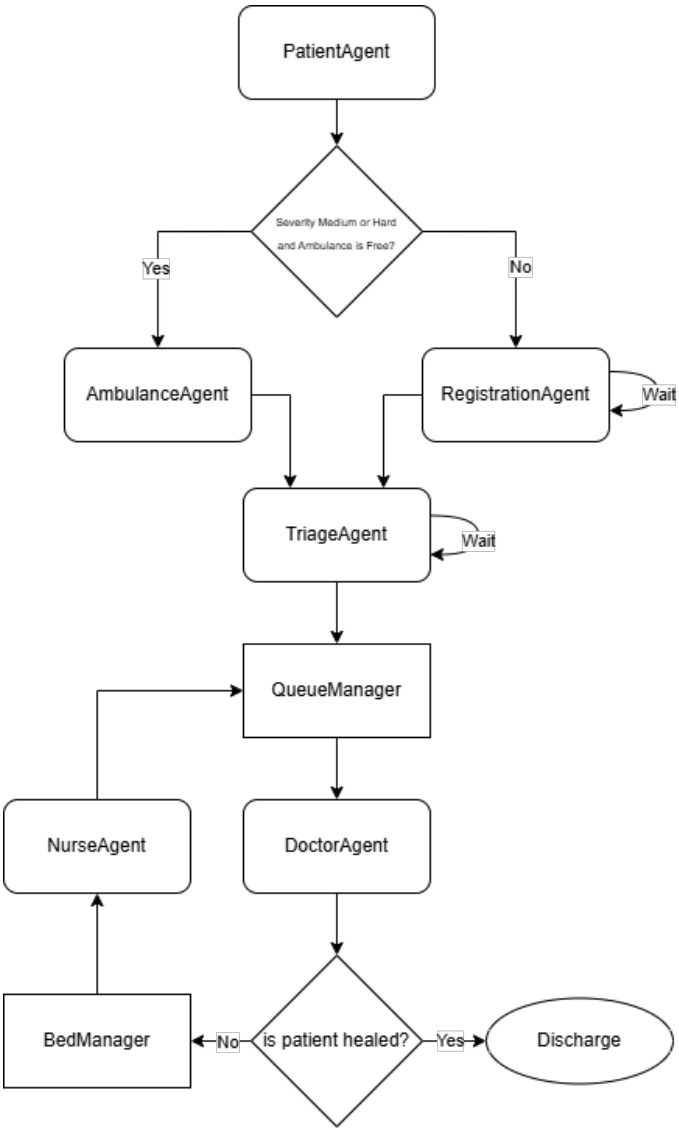

Figure 1: Workflow and interaction diagram of the Emergency Department MAS.

As shown in the diagram, the system manages the patient flow through three main stages:

4.1 Patient Intake and Triage

The process begins when a **PatientAgent** is spawned. The system evaluates a branching condition based on the patient's severity and the availability of resources:

- **Ambulance Path:** If the severity is Medium or High and an **AmbulanceAgent** is available, the patient bypasses manual registration.

- **Walk-in Path:** Otherwise, the patient must interact with the **RegistrationAgent**, potentially waiting if the desk is busy.

Both paths converge at the **TriageAgent**, which assigns a clinical priority color and inserts the patient's data into the **QueueManagerArtifact**.

4.2 Medical Examination and Treatment

The **DoctorAgent** instances act as the primary consumers of the priority queue. A doctor retrieves the next patient from the **QueueManagerArtifact** and performs the treatment.
The outcome of the examination leads to a clinical decision based on the updated triage color:

- **Discharge:** If the patient's condition is stable (Triage Color: White) or if the doctor determines that hospitalization is not required despite a higher color code (e.g., Green or Blue), the patient is **Discharged**.
- **Hospitalization:** If the patient requires continuous monitoring or further stabilization, the doctor attempts to secure a bed through the **BedManagerArtifact**. This decision is based on a probability distribution tied to the current triage color.

4.3 Monitoring and Re-evaluation Loop

Patients assigned to a bed enter a monitoring phase managed by the **NurseAgent**. This agent periodically checks the patient's status via the **BedManagerArtifact** and, when appropriate, re-inserts the patient into the **QueueManagerArtifact**. This creates a feedback loop that ensures hospitalized patients receive follow-up examinations by a **DoctorAgent** until they are ready for discharge.

5 User Interface

The simulation is controlled and monitored through two distinct graphical interfaces, designed to separate the setup phase from the real-time execution.

5.1 Configuration Interface

The **Configuration UI** is the initial window presented to the user. It serves as the control panel for initializing the Multi-Agent System by allowing the adjustment of key parameters.

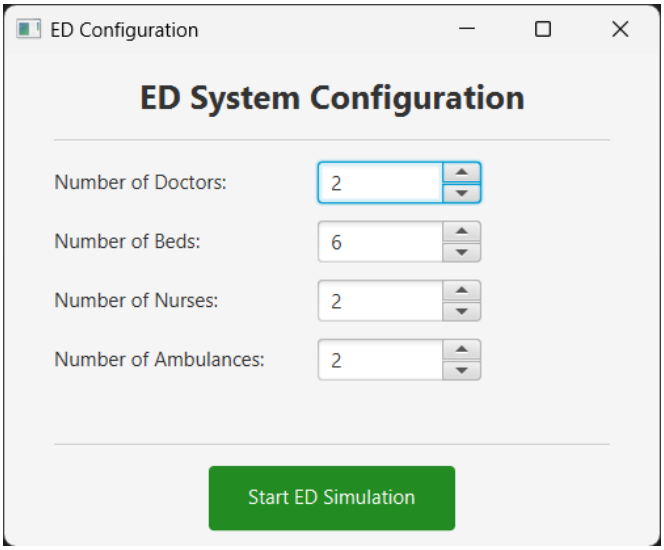

Figure 2: Initial configuration window for simulation parameters.

Upon clicking the "Start Simulation" button, the system initializes the agents and artifacts with the specified parameters and opens the **Main UI**.

5.2 Main Simulation Dashboard

The **Main UI** provides a real-time overview of the Emergency Department's status, organized into a four-quadrant grid layout to maximize information density.

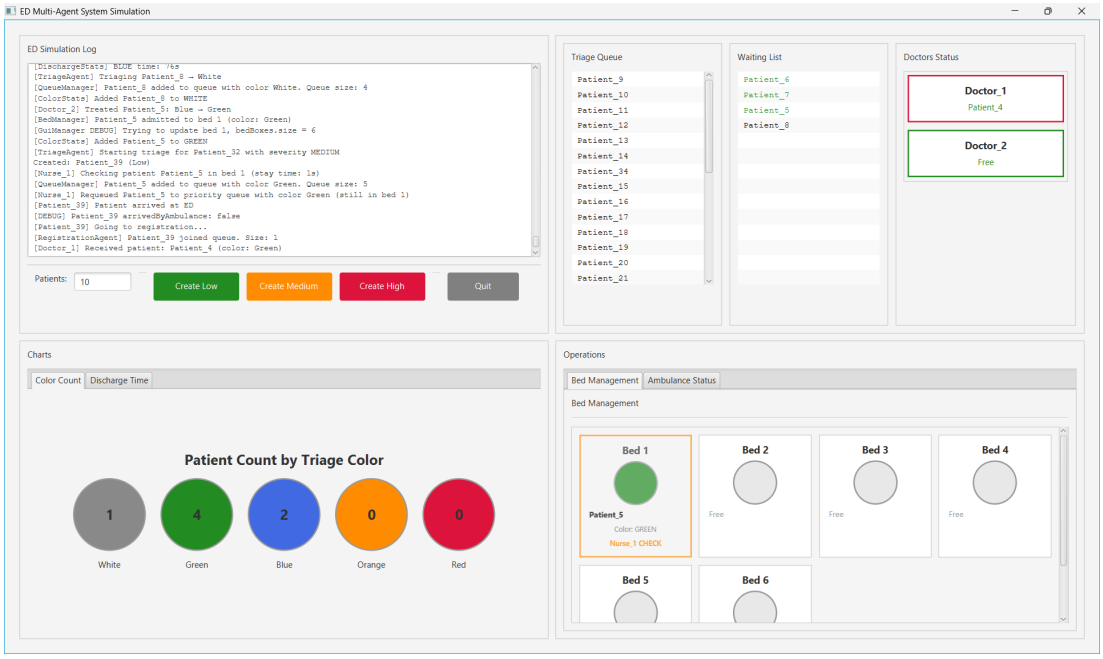

Figure 3: Full overview of the Emergency Department Dashboard.

To better illustrate the specific functionalities, the following figures detail the individual components of the dashboard:

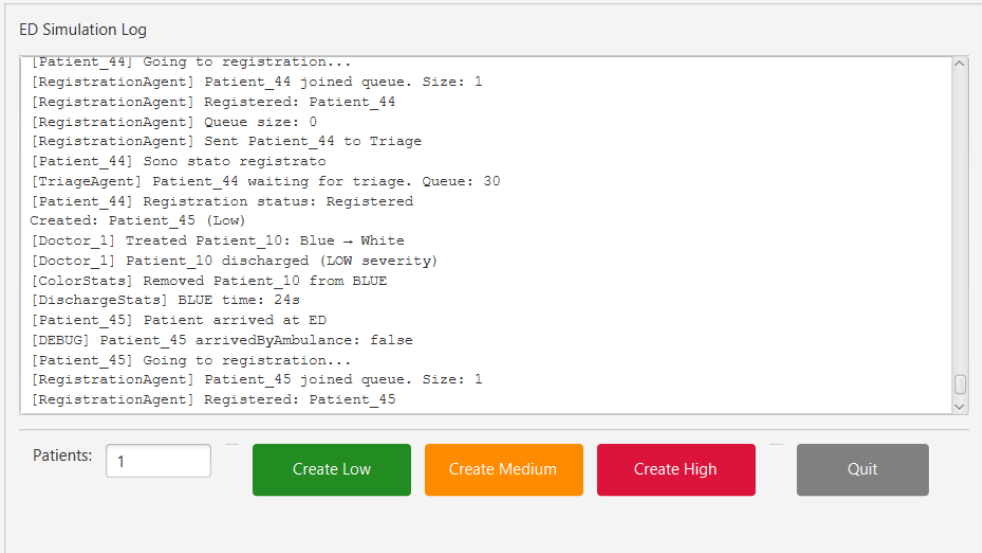

Figure 4: Detailed view: Event Log and Manual Controls.

The **Event Log** (Figure 4) displays a real-time feed of significant system events, such as patient arrivals, triage assignments, doctor-patient interactions, and bed occupancy changes. This log is crucial for monitoring the simulation flow and diagnosing potential issues. It also includes manual controls to spawn a custom number of patients with specific severity levels and a "Quit" button to terminate the simulation.

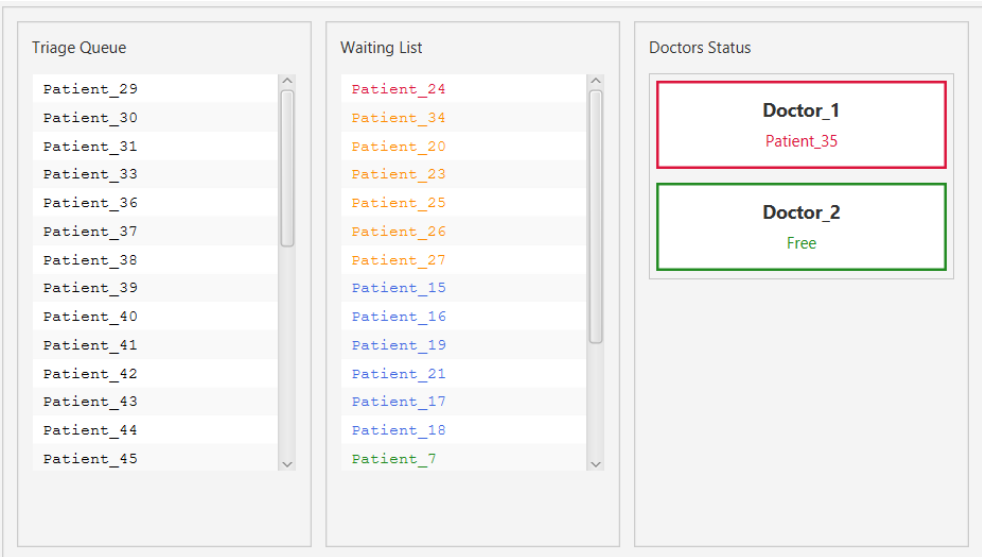

Figure 5: Detailed view: Triage/Priority Queues and Doctor Status.

The **Waiting Queues and Doctor Status** section (Figure 5) provides a snapshot of the current state of the Triage and Priority Queues. It also lists all active **DoctorAgent** instances, indicating their current availability and, if busy, the ID of the patient they are attending to.

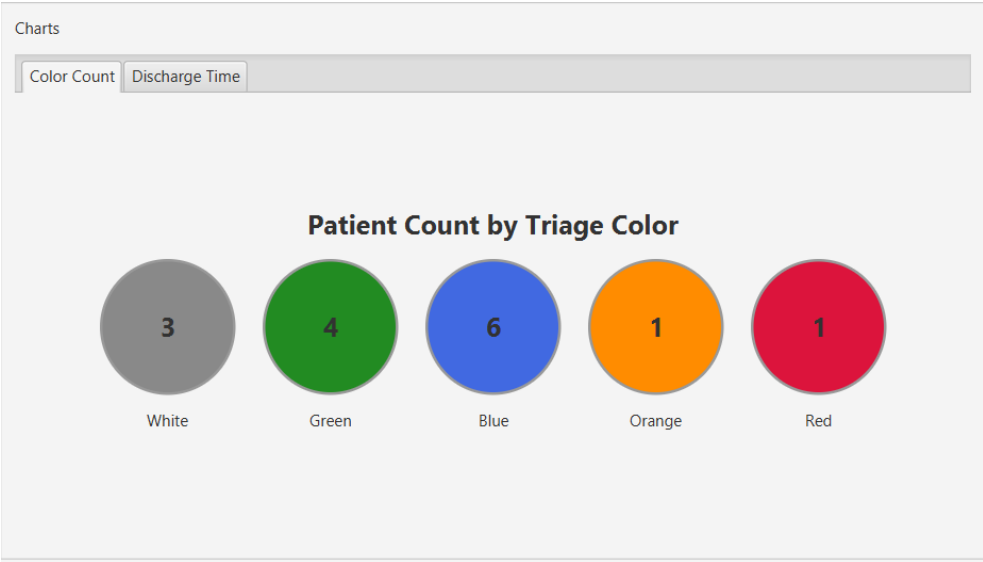

Figure 6: Detailed view: Real-time Triage Color Distribution Chart.

The **Triage Color Distribution Chart** (Figure 6) dynamically visualizes the proportion of patients across different triage colors. Inspired by the official triage monitoring systems used in the Liguria region [5], this chart updates in real-time as patients are triaged or treated, providing an immediate overview of the case-mix handled by the department according to standard clinical priority scales.

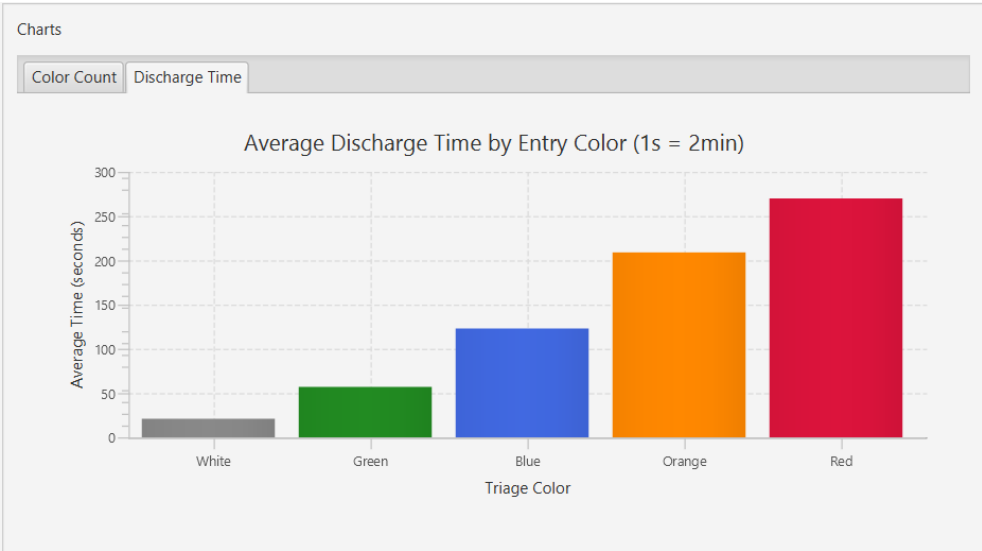

Figure 7: Detailed view: Average Discharge Time Statistics Chart.

The **Average Discharge Time Chart** (Figure 7) tracks the time patients spend in the system before being discharged, categorized by their entry triage color. This metric is essential for evaluating the efficiency of the treatment process and identifying potential bottlenecks.

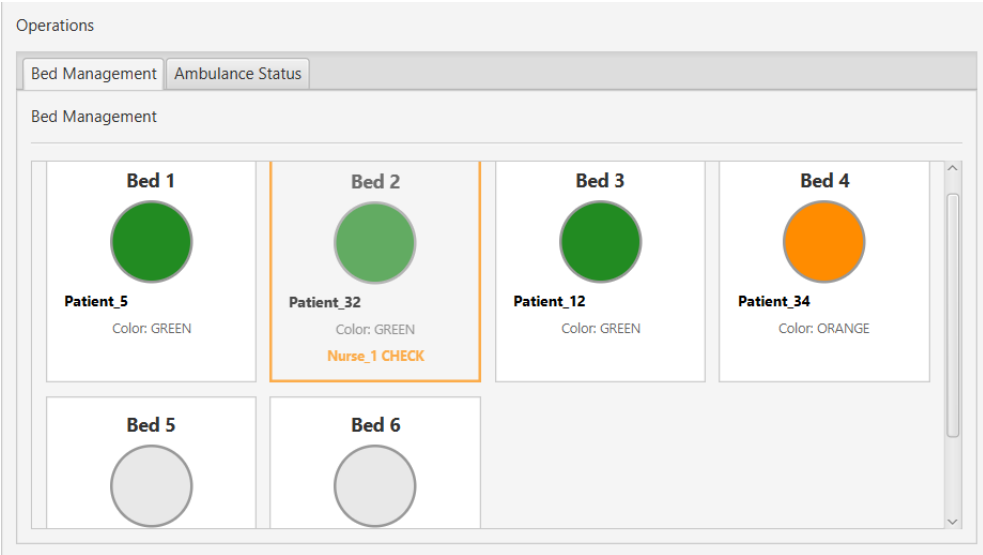

Figure 8: Detailed view: Bed Occupancy and Nursing Status.

The **Bed View** (Figure 8) offers a visual representation of all beds. Each bed is color-coded based on the occupant's triage level, with additional indicators showing if a **NurseAgent** is currently performing a check. This allows for a quick assessment of resource allocation.

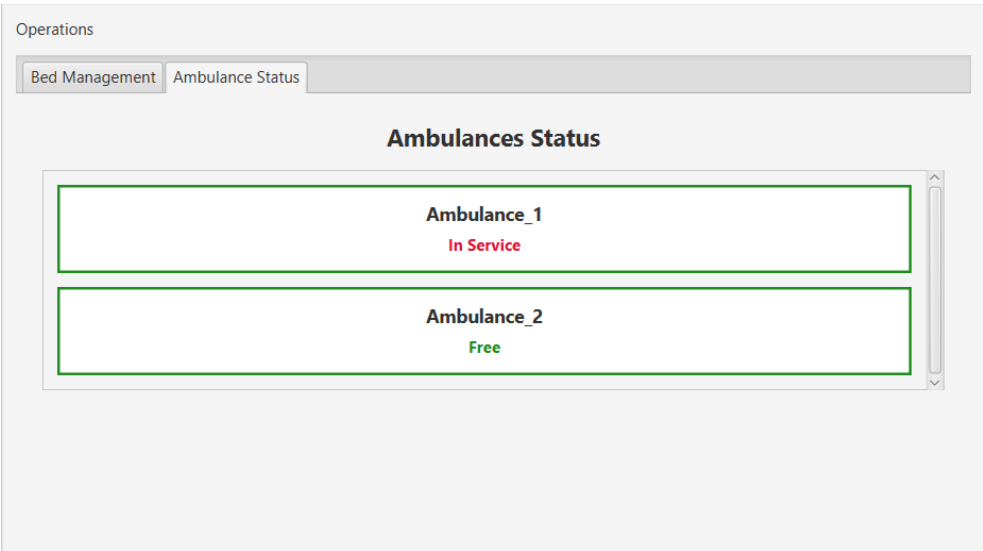

Figure 9: Detailed view: Real-time Ambulance Fleet Monitoring.

Finally, the **Ambulance Fleet Monitoring** section (Figure 9) provides real-time status updates for all **AmbulanceAgent** instances, indicating whether they are currently on a mission or available for a new dispatch.

Note. All screenshots were taken using the default simulation configuration: 6 beds, 2 doctors, 2 nurses, and 2 ambulances.

6 Code Specification and Implementation

This section details the technical implementation of the Emergency Department MAS, including the libraries used, the concurrency management strategies, and the overall project structure.

6.1 Development Stack and Libraries

The system is built using **Java 21** and managed through **Apache Maven** [1]. Maven was chosen to handle the project's lifecycle, ensuring consistent builds and managing external dependencies. The core framework leverages the following components:

- **JADE (v4.5.0):** The primary framework for agent management and FIPA-compliant ACL messaging [6].
- **JavaFX (v21.0.2):** Used for the graphical user interfaces and real-time visualization of hospital statistics [3].
- **SLF4J:** Implemented for robust event logging and debugging across the multi-agent system.
- **Standard Java Concurrency:** Specifically the `ReentrantReadWriteLock` to ensure thread-safety within the shared artifacts.

The build process is automated to generate an executable *fat-jar* using the `maven-shade-plugin`, simplifying the deployment and execution of the simulation.

6.2 Concurrency and Synchronization

A critical challenge in this multi-agent system was managing simultaneous access to shared resources, such as the **QueueManager** and the **BedManager**. To prevent *race conditions* while maintaining high performance, the `java.util.concurrent.locks.ReentrantReadWriteLock` was implemented.
Unlike a standard `synchronized` block, this lock allows:

- **Multiple concurrent readers:** Multiple **DoctorAgent** or **NurseAgent** instances can check bed or queue status simultaneously without blocking each other.
- **Exclusive writing:** When an agent (e.g., a **TriageAgent**) needs to add a patient or update a triage color, it gains exclusive access to the data structure, ensuring atomicity and consistency.

6.3 Project Structure and Packages

The source code is organized into a modular package structure to ensure separation of concerns:

- `it.unige.dibris.mas.agents`: Contains the logic for all agents (*Patient, Doctor, Nurse, etc.*).
- `it.unige.dibris.mas.behaviours`: Defines the specific JADE behaviors, such as `TickerBehaviour` for periodic monitoring and `CyclicBehaviour` for message handling.
- `it.unige.dibris.mas.ontology`: Defines the data structures and enumerations (e.g., `TriageColor`, `PatientSeverity`, `BedInfo`).
- `it.unige.dibris.mas.gui`: Manages the JavaFX components and the `SimulationLogger` for real-time event reporting.

6.4 Metrics and Repository

The final implementation consists of approximately **2880** lines of code, distributed across **30** Java files.
The complete source code, including the configuration files and documentation, is available at the official repository [2]:

`https://github.com/matteogrifone22/sdai-project.git`

6.5 Use of Artificial Intelligence and LLMs

LLMs and Generative AI were utilized as auxiliary tools for the following tasks:

- **Code Generation and Setup:** To automate the writing of repetitive structural components and to assist in the initial setup of the Maven project structure.
- **Syntax and UI Design:** To provide guidance on **JavaFX** syntax and CSS-like styling for the graphical user interface.
- **Language and Documentation:** To perform grammatical and stylistic checks throughout this report.

7 Future Improvements

While the current simulation provides a robust representation of Emergency Department dynamics, it should be considered a foundational model that remains open to several functional enhancements. A primary direction for future development could involve increasing the granularity of the clinical workflow, for instance, the **DoctorAgent** logic could be expanded to distinguish between the initial medical examination and subsequent diagnostic phases, such as specialized tests or imaging. Similarly, introducing professional specializations among medical staff would allow for a more realistic assignment of cases based on patient severity. Furthermore, the analytical depth of the system could be improved by collecting and displaying more sophisticated real-time statistics, such as resource utilization rates or detailed waiting time distributions. These additions would transform the current framework from a general simulation into a highly precise decision-support tool for hospital management.

References

[1] Apache Software Foundation: Apache maven project (2024), `https://maven.apache.org/`

[2] Ferrari, M.: Emergency department multi-agent system (ed-mas) source code. `https://github.com/matteogrifone22/sdai-project.git` (2026)

[3] Gluon: Javafx documentation (2024), `https://openjfx.io/`

[4] Jones, S.S., Evans, R.S.: An agent-based simulation tool for scheduling emergency department physicians. In: AMIA Annual Symposium Proceedings. vol. 2008, p. 338. American Medical Informatics Association (2008), `https://pmc.ncbi.nlm.nih.gov/articles/PMC2656074/`

[5] Regione Liguria: Monitoraggio tempo reale pronto soccorso - liguria (2026), `https://pslive.regione.liguria.it/`

[6] Telecom Italia S.p.A.: Jade programming guide (2017), `https://jade.tilab.com/doc/programmersguide.pdf`